

Do LLMs Silently Compute the Wrong Thing? Measuring and Detecting Silent Semantic Errors in LLM-Generated Data-Transform Code with Goldless Operator Contracts

Anonymous Submission

Abstract

Large language models increasingly write the *data-transform* code behind analyses and figures—group-wise medians, weighted means, joins, pooled rates. We show that a large fraction of this code is *silently* wrong: it runs without error and produces plausibly-shaped output, yet computes a different quantity than the user asked for. On real Nature source-data tables, frontier models emit such silent semantic errors in **77%** of ambiguous data-transform tasks, and the failure is *invisible* to every simple safeguard—deterministic checks (execution-passes, shape, value-range, self-consistency) catch **0%**, and LLM self-critique misses 39% while false-flagging 40%. We then introduce *goldless typed operator contracts*: executable invariants over the (input, parameters, result) triple that certify an operator’s semantics *without any reference answer*. On real Nature data the contracts detect silent errors at **99% recall / 0% false-positive** ($N=1408/1539$). We show the contracts can be *auto-synthesized* from a high-level natural-language goal (78% correct from goal-only intent, $N=23$, one-shot), transfer across execution substrates unchanged (pandas→SQL, 10/10 operators), and drive a zero-training detect→localize→repair recipe (targeted repair 79% vs. 5% for strong self-debug). Finally, we run the *entire* pipeline end-to-end from only a messy analyst request and a raw table—inferring the operator, grounding column roles, synthesizing the contract, and detecting the error—and catch 51–58% of silent errors on real Nature tables with nothing structured provided. We release the measurement suite, the contract library, and the synthesis code.

1 Introduction

Code-writing assistants now sit in the middle of everyday data analysis: a user describes what they want in plain language (“give me the typical expression per cell type”), and the model returns pandas (or SQL) that computes it. When that code has a *syntactic* bug it crashes, and when it has an obvious numeric bug the plot looks wrong. The dangerous case is the third one: the code *runs*, the output has the right shape and a believable range, and it is nonetheless the answer to a *different question*—a mean where the user meant a median, a per-row ratio averaged instead of a pooled rate, an

inner join that silently drops unmatched rows. We call these **silent semantic errors**.

Silent semantic errors are pernicious precisely because the standard safety net does not see them. Execution-based evaluation (“does it run?”), shape/type validity, and self-consistency across samples all pass, because the wrong code is still *valid* code producing *valid-looking* output. Verifying the *meaning* of the result normally requires a gold reference—but in deployment there is no gold: if we already knew the right answer we would not be asking the model.

This paper. We make three moves. **(1) Measurement.** We quantify how often frontier LLMs silently compute the wrong data transform, on *real* scientific data (Nature source-data tables) and across models, and we quantify how completely existing checks miss it. **(2) A goldless detector.** We introduce *typed operator contracts*: small executable invariants of an operator’s semantics—conservation laws, per-group identities, row-count and range constraints—checked over the (input, params, result) triple with *no reference answer*. **(3) From assertions to a system.** We show the contracts are not a hand-written wall of `asserts` but the core of an AI pipeline that *grounds an informal request into a verifiable constraint*: it auto-synthesizes the invariant from a natural-language goal, infers the operator and column roles from a messy request, refines the contract against a goldless consensus oracle, and repairs the code—end to end.

Contributions.

1. **A measurement study** of silent semantic errors in LLM data-transform code: 77% on real Nature tasks, 32–78% across ten models/four vendors, 26% on external DS-1000; deterministic checks detect 0%, self-critique 61% recall at 40% false-positive (Sec. 4).
2. **Goldless typed operator contracts** that detect the errors at 99% recall / 0% false-positive on real Nature data ($N=1408/1539$), where the best code-layer baselines are structurally blind (Sec. 5–6).
3. **Automatic contract synthesis** from a high-level NL goal (78% correct from goal-only intent, 83% when a formula is given, $N=23$, one-shot; stable across GPT and Gemini), plus a monotone-safe counterexample-guided *goldless* refinement loop.
4. **An end-to-end zero-template pipeline**—infer operator, ground column roles, synthesize contract, detect—run on

real Nature tables from only a messy request and a raw table: 51–58% of silent errors caught with nothing structured given.

5. **Substrate-agnostic transfer** (the same contracts fire on pandas and SQL silent slips, 10/10 operators, zero SQL-specific code) and a **zero-training repair recipe** (targeted repair 79% vs. 5% strong self-debug vs. 45% localization-only).
6. **Honest boundaries, quantified**: the method’s power lives where operator-level intent is recoverable; on raw free-text code operator inference is the bottleneck, and a calibrated scope-gate makes out-of-scope inputs *safely abstain* (Sec. 7).

2 Related Work

Evaluating LLM code generation. Benchmarks such as HumanEval, MBPP, and DS-1000 [1, 2, 3] score functional correctness against *hidden test cases* or a *gold reference*. Execution-based agent evaluation [4] likewise assumes a checkable oracle. Our setting is complementary and harder: no gold reference exists at inference time, and the code already passes execution.

Data-science and visualization agents. Chart/plotting benchmarks (MatPlotBench, Plot2Code, ChartMimic) [11, 12, 13] and code-similarity or visual-similarity judges [14] score whether the figure *looks* right, not whether the plotted numbers *equal* the data. We measure and detect the upstream data-transform error that such judges are structurally unable to see.

Design-by-contract and property-based testing. Runtime contracts [5] and property-based testing [6] are decades old. Our contribution is not the mechanism of asserting a property but (i) identifying an *empirically-grounded taxonomy* of silent operator-level slips in LLM output, (ii) showing the invariants can be *goldless* (need no reference answer) and *auto-synthesized from natural language*, and (iii) grounding an informal user request into the right invariant automatically.

Validating generations without gold. Self-consistency [8], self-critique, and LLM-as-judge [9, 10] estimate correctness without a reference but, as we show, miss silent semantic errors (61% recall / 40% FP). Our consensus + metamorphic oracle for contract refinement is a goldless verification signal in the spirit of metamorphic testing [7].

3 Problem Formulation

A *data-transform task* is a triple (D, ι, θ) : an input table (or tables) D , a natural-language intent ι , and column-role parameters θ (which column is the group, the value, the weight,

Safeguard	Recall	False-positive
Execution-passes / shape / range / consistency	0%	0%
LLM self-critique	61%	40%
Goldless operator contracts (ours)	99%	0%

Table 1: Detection of silent semantic errors on real Nature data ($N=1408$ recall / 1539 false-positive). Simple deterministic checks are structurally blind; self-critique is too noisy to gate on.

...). A model produces code f whose result is $r = f(D)$. Let g denote the intended (gold) transform. A **silent semantic error** occurs when f executes without error, r is schema-valid, yet $r \neq g(D)$ up to tolerance.

We target the **goldless** regime: at inference time g is unavailable. A **typed operator contract** for an operator o is a function $C_o(D, \theta, r) \in \{\text{fire}, \text{pass}\}$ that encodes a *checkable invariant* of o ’s semantics and *fires* iff r violates it. A good contract satisfies two properties, measured against held-out correct and slip implementations: it *fires on the silent slip* (recall) and *passes on every valid implementation*, including alternative correct ones (precision / false-positive robustness). Crucially C_o inspects only (D, θ, r) —never the code f —so it is agnostic to how r was produced.

4 Measurement: How Common, How Invisible

Corpus. We pair Nature figures with their Source-Data tables and instantiate operator-semantic tasks (median-vs-mean, within-group share, weighted mean, pooled rate, NaN-as-zero, ...) on the real tables, so the table, the columns, and the ground-truth transform are *not* authored by us.

Prevalence. Prompted with a realistically *ambiguous* request, frontier models emit a silent semantic error on **77%** of real Nature tasks. The rate is **32–78%** across ten models and four vendors, and **26%** on external DS-1000 data-frame tasks—so this is a property of the task family, not one model. [\[FIG: prevalence bar chart across models\]](#)

Invisibility. On the same corpus we measure what existing safeguards catch (Table 1). Deterministic checks—execution-passes, shape/type validity, value-range, and multi-sample consistency—catch **0%**: the wrong code runs, is shaped correctly, and is *stably* wrong. LLM self-critique catches 61% but false-flags 40%, which is unusable as a gate.

5 Method: Goldless Contracts as an AI Pipeline

5.1 Typed operator contracts

Each operator is certified by one small invariant, e.g.: a *weighted mean* must lie within the min/max of the values and move toward the higher-weighted ones; *within-group shares* must sum to one *per group* (not globally); a *pooled rate* equals $\sum \text{num} / \sum \text{den}$, not the mean of per-row ratios; a *left join* preserves every left row. Each contract is ≈ 10 lines and reads only (D, θ, r) .

5.2 Auto-synthesizing contracts from natural language

To show the taxonomy is not a closed hand-written set, we ask a frontier model to *synthesize* the contract from only the NL goal, the parameter names, and the result schema—it never sees the reference implementation. To rule out the model merely *translating a stated formula*, we run a *de-leaked* condition whose intent gives only the high-level goal (no formula, no “not-the-X”, no invariant). Synthesis stays at **78%** correct (de-leaked) vs. 83% (formula-given), $N=23$, one-shot—so the model is *deriving* the invariant, not transcribing it (Sec. 6.2).

5.3 Goldless counterexample-guided refinement

Auto-synthesis can over-fire on valid alternative implementations. We add a monotone-safe refinement loop with a *goldless* oracle: sample K diverse implementations of the intent, cluster by output to form a *consensus* (probably-correct) set, and treat any contract that fires on a consensus member as a false-positive counterexample; *metamorphic* perturbations of a consensus result probe non-triviality. A refinement is accepted only if a goldless score (consensus-pass + probe-fire) strictly improves, so the loop never regresses below one-shot by construction.

5.4 End-to-end pipeline from a messy request

Deployed, the system receives only a messy analyst request ι and a raw table D . It (1) infers the operator $\hat{o}$ from ι , (2) grounds the column roles $\hat{\theta}$ from ι and the column names, (3) synthesizes the contract $C_{\hat{o}}$, and (4) checks r . Nothing is template-given.

5.5 Detect $\rightarrow$ localize $\rightarrow$ repair

When a contract fires, the same goldless signal drives repair: the contract’s violated invariant is fed back to the model as a *targeted* repair prompt. Best-of- N selection uses the contract (not gold) to pick among candidate fixes.

Intent given to synthesizer	CORE	FULL
Formula-stating (“leaky”)	83%	78%
High-level goal only (de-leaked)	78%	70%

Table 2: One-shot goldless contract synthesis, $N=23$ operators. CORE = fires-on-slip and passes-on-correct; FULL adds robustness to alternative valid implementations. Only -5 points when the formula is removed: the model *derives* the invariant.

6 Experiments

Unless noted, the frontier model is GPT-5.4 behind an OpenAI-compatible API; all intervals are 95% Wilson. Buggy starting points for the repair experiments are *model-generated*, not author-written.

6.1 Detection on real Nature data

The contracts detect silent errors at **99% recall** (1398/1408) and **0% false-positive** (2/1539) on real Nature tables—the same corpus used for measurement, so the tables, columns, and gold are external (Table 1). We distinguish this from a self-authored correctness grid, which we use only as a contract *unit test*, not as evidence of detection generalization.

6.2 Automatic contract synthesis

Table 2 reports one-shot synthesis over 23 operators. The de-leaked drop is only 5 points (83 $\rightarrow$ 78 CORE), and the result is stable across GPT-5.4, GPT-5.5, and (different vendor) Gemini-3.1-Pro at 83%, ruling out model cherry-picking. The goldless CEGIS loop is monotone-safe and neutral on the already-strong one-shot (both 83% CORE, $N=12$)—we report this honestly: its value is a *goldless verification algorithm*, not an accuracy boost; residual failures are recall-type, which false-positive refinement cannot address.

6.3 Recovering the operator from natural language

Given a realistic request with *no operator keyword*, the frontier model recovers the operator on real Nature tasks at **98%** (147/150), while a keyword regex collapses to **21%** (31/150)—confirming the model uses semantics, not string matching (the keyword-stuffed upper bound is 100% for both). End-to-end detection retention under inferred (vs. given) operators is 98%.

6.4 End-to-end from only a messy request and a raw table

Table 3 runs the full pipeline with nothing template-given. On generated fixtures the full system catches 61%; on *real* Nature tables 58%; and when even the column-role parameters are inferred from the request (so the input is *only*

Setting	Op infer	Contract	Full system
Synthetic fixtures	83%	74%	61%
Real Nature tables	76–83%	70–74%	58%
+ inferred column roles	params 74% (83% clean)		51%

Table 3: End-to-end zero-template detection ($N=23$ fixtures; $N=33/35$ real tables). The last row provides *only* a messy request and a raw table—operator, column roles, and contract are all inferred.

a messy sentence and a raw table), 51%—just 3 points below the params-given setting. Parameter grounding is 83% on type-disambiguated roles (categorical group vs. numeric value); the lower overall figure comes from genuinely symmetric roles (weighted mean’s value-vs-weight, pooled rate’s numerator-vs-denominator) that the request does not uniquely fix. We report 51% as a *useful early-warning* floor, not a near-deployable figure: the deployable regime is the structured-intent 99%/0%.

6.5 Substrate-agnostic transfer (pandas $\rightarrow$ SQL)

Because a contract reads only (D, θ, r) , the *same* contracts fire on the identical semantic slip written in SQL. Over 10 operators (weighted AVG, WHERE g IS NOT NULL, join fan-out, inner-join drop, AVG of ratios, missing DISTINCT, COUNT(`col`) vs COUNT(`*`), window OVER() vs PARTITION BY, ...), the contracts fire on every SQL slip and pass every correct SQL, with **zero** SQL-specific code (10/10).

6.6 Zero-training repair

Feeding the violated invariant back as a targeted repair prompt fixes **79%** (34/43) of silent errors, versus **5%** (2/43) for a strong self-debug baseline (per-value re-check + self-critique) and 45% for localization-only—the invariant contributes a large, statistically separated gain (disjoint CIs). Best-of- N selection using the contract (no gold) lifts repair further with zero mis-repairs.

7 Limitations and Honest Boundaries

Conditional validity. The method’s strength lives where operator-level intent is recoverable. On *raw free-text code* (DS-1000), inferring which operator to check is the bottleneck and no operating point reaches usable recall/precision; a calibrated LLM scope-gate instead makes out-of-scope inputs *abstain*, cutting spurious firing from 55% to 5%. The deployable claim is therefore “NL intent recoverable,” not “arbitrary code.” **Parameter symmetry.** For operators with symmetric numeric roles the request may not fix which column is which; we report this as ambiguity, not a method failure. **End-to-end ceiling.** The 51–58% zero-structure figure

is an early-warning floor, well below the 99%/0% structured-intent regime. **CEGIS is neutral** on the strong one-shot synthesizer; we include it as a goldless *algorithm*, not an accuracy gain. **Scope.** Results are on tabular data transforms (pandas/SQL); other substrates are future work.

8 Conclusion

LLMs silently compute the wrong data transform far more often than execution- or shape-based checks reveal (77% on real Nature data; 0% caught by simple safeguards). Typed operator contracts detect these errors without any gold reference (99%/0%), can be auto-synthesized from a high-level request, transfer across execution substrates, and drive a zero-training repair recipe—and the whole pipeline runs end-to-end from only a messy request and a raw table. We frame silent semantic error detection as grounding an informal intent into a verifiable, goldless constraint, and release the measurement suite and code.

References

- [1] M. Chen et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
- [2] J. Austin et al. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021.
- [3] Y. Lai et al. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. ICML, 2023.
- [4] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024.
- [5] B. Meyer. Applying Design by Contract. IEEE Computer, 25(10):40–51, 1992.
- [6] K. Claessen and J. Hughes. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. ICFP, 2000.
- [7] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic Testing. Tech. Rep. HKUST-CS98-01, 1998.
- [8] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.
- [9] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023.
- [10] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023.
- [11] Z. Yang et al. MatPlotAgent: Method and Evaluation for LLM-Based Agentic Scientific Data Visualization. ACL Findings, 2024.
- [12] C. Wu et al. Plot2Code: A Comprehensive Benchmark for Evaluating MLLMs in Code Generation from Scientific Plots. arXiv:2405.07990, 2024.
- [13] C. Shi et al. ChartMimic: Evaluating LMM’s Cross-Modal Reasoning via Chart-to-Code Generation. arXiv:2406.09961, 2024.
- [14] N. Chen et al. VisEval: A Benchmark for Data Visualization in the Era of Large Language Models. IEEE TVCG, 2024.